



**Hochschule
Kaiserslautern**
University of
Applied Sciences

Angewandte
Ingenieurwissenschaften
Kaiserslautern

Course of studies:

Master of Mechanical engineering / Mechatronics

Project Title:

System level rapid development in mechatronics

Submitted by:

Konda Madana Gopala Reddy
Matriculation No.: 889670

Mentor:

Prof. Dr.-Ing. Gerd Bitsch

Table of Contents

8.Challenges to Master	4
8.1 Challenge 1 – Driving Fast	4
8.2 State-Feedback Velocity Controller	4
8.3Position–Velocity State-Feedback Controller	6
8.2. Challenge 2 – Driving fast with turns	9
8.2.1 Stateflow logic for distance- and yaw-based motion control	9
8.2.2 PID-based control using conditional switching	11
.....	11
Stateflow logic for distance- and yaw-based motion control	13
8.3. Challenge 3 – Additional weight compensation	13
8.3.1. Steady-state error control.....	13
8.3.2. PID controller implementation	14
8.3.3. Cascaded PID control	14
Real time implementation: - realtime implementation of weight balance on oneside	15
8.4 Driving a figure 8.....	16
8.4.1 Initial approach: time-based figure-8 motion	16
8.4.2 Second iteration: yaw-driven figure-8 control.....	17
8.4.3 Final iteration: figure-8 control using position feedback	18
8.5. DIP switch control	20

List Of Figures

Figure 8-1 Mean forward velocity response for the three-meter sprint.....	4
Figure 8-2 Mean estimated velocity and corresponding time marker	5
Figure 8-3 Stateflow-based phase logic for velocity ramp-up.....	5
Figure 8-4 Velocity reference generation from distance error	6
Figure 8-5 Truth-table–based velocity selection.....	7
Figure 8-6 Truth table conditions	7
Figure 8-7 Phase-scheduled State flow logic.....	7
Figure 8-8 Mean forward velocity response using the position–velocity controller.....	8
Figure 8-9 Stateflow sequence for fast forward motion with two 360° turns	9
Figure 8-10 Complementary filter for pitch and yaw estimation	10
Figure 8-11 Response of the robot at different position targets.....	10
Figure 8-12 Stateflow logic for fast forward motion with two controlled 360° turns.....	11
Figure 8-13 PID control that switches between driving and holding position	11
Figure 8-14 Final position PD controller with state-based switching	12
Figure 8-15 Robot response to step-wise position and velocity commands.....	12
Figure 8-16 Design of the steady-state error compensation controller.....	13
Figure 8-17 Position error PID control architecture	14
Figure 8-18 Cascaded PID structure for position and velocity regulation	14
Figure 8-19 Test loads used for balance experiments.....	15
Figure 8-20 Stateflow-based time-driven logic for the figure-8 path.....	16
Figure 8-21 Complementary-filter-based pitch and yaw estimation code.....	17
Figure 8-22:-configuration for figure 8	18
Figure 8-23 Position-referenced Stateflow logic for figure-8 trajectory	18
Figure 8-24 final controller for driving figure 8	18
Figure 8-25. Simulink-based DIP switch interface for mode control.....	20
Figure 8-26. Truth-table based mode selection in Simulink.....	20
Figure 8- 27 Logic table for selecting the operating mode.....	21
Figure 8-28 Combined controller design for all 4 challenges with control switch for logic selection.....	21

List of Tables

Table 1:-Feedback table for velocity controllor.....	6
Table 2:-summary of the position–velocity state-feedback controller across multiple trials	8
Table 3:-Summary of all the controllers for challenge 1	9
Table 4:- Position deviation comparison for different control strategies.....	13
Table 5:- Cascaded PID gains and the corresponding balanced weights on both sides.....	15
Table 6:-Cascaded PID gains and balanced weight with load applied on one side	15
Table 7:-Parameters and step-by-step calculation for circular motion	16
Table 8:-Impact of forward velocity and timing on the figure-8 path	17
Table 9:- Evaluation of motion control approaches for the figure-8 trajectory	19
Table 10:-Use of AI tools in this project.....	22

8.Challenges to Master

8.1 Challenge 1 – Driving Fast

In this task, the robot is required to traverse a distance of three meters as quickly as possible. A run-up before the measured section is allowed, enabling the system to build momentum early and operate at higher velocities. Reaching this regime demands careful manual tuning of the state feedback gains, with stability becoming increasingly fragile as speed increases. Each gain influences the robot in a different way. Increasing the velocity gain improves acceleration and responsiveness, but also pushes the system closer to instability. Higher pitch gain reduces pitch angle error more aggressively, yet amplifies oscillations when overemphasized. The pitch rate gain provides damping; near its optimal value it suppresses oscillatory motion, while deviations on either side lead to degraded behaviour. High-speed performance therefore emerges only from a balanced tuning of all feedback gains. Pushing a single parameter is ineffective. Speed is gained only as long as stability can still be maintained.

8.2 State-Feedback Velocity Controller

The first controller implementation is based on a velocity-driven state feedback structure combined with a phase-based motion logic. At its core, the controller stabilizes the robot using full state feedback while progressively increasing translational velocity through a structured sequence of operating phases rather than a single step command.

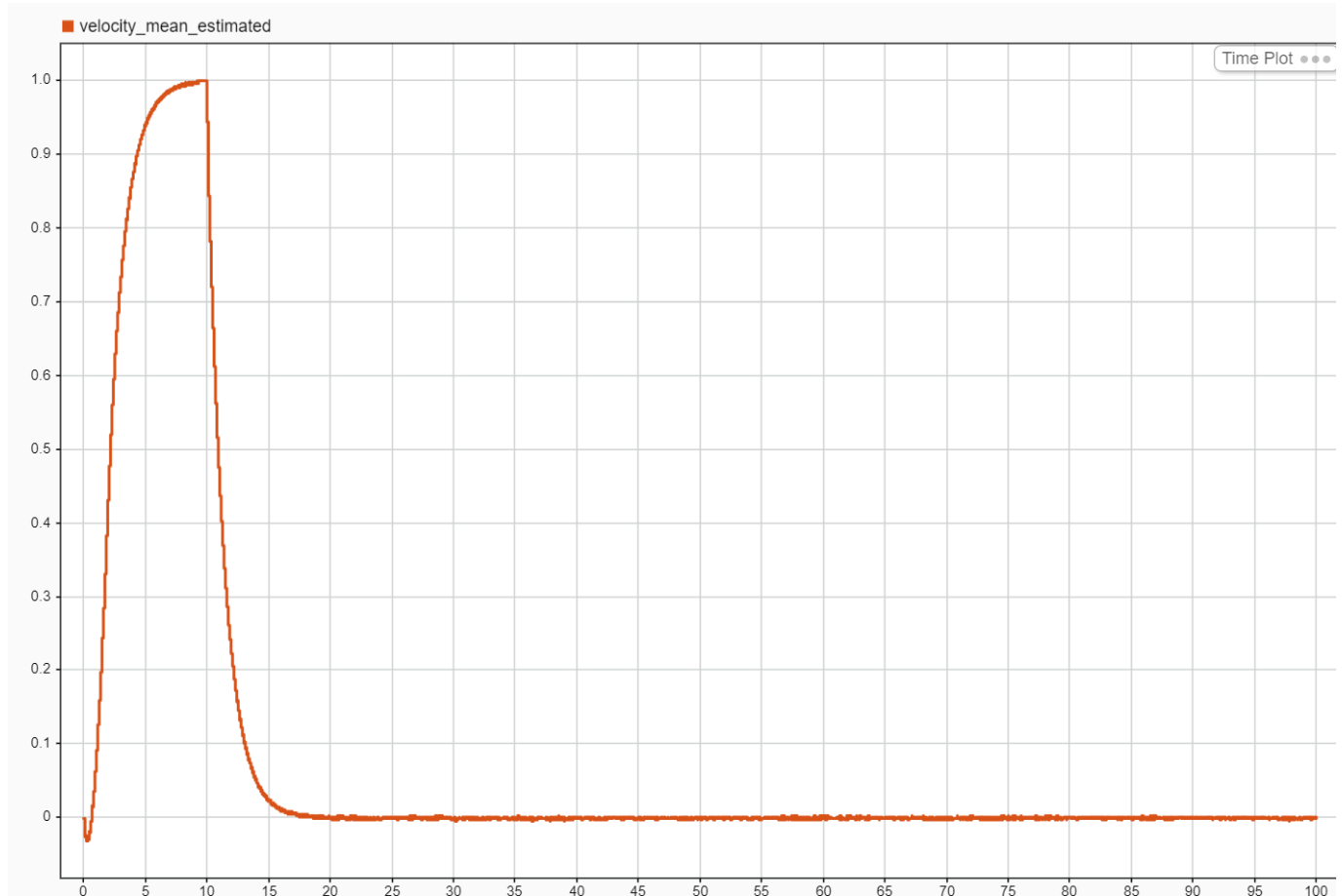


Figure 8-1 Mean forward velocity response for the three-meter sprint

The velocity response obtained with the state feedback gain set $[-3, 20, 1.5]$ is shown in the figure 8-1. With these gains, the robot reaches a peak velocity of approximately 1 m/s, without loss of balance in virtual simulation. However, the response is clearly conservative. The velocity builds up slowly and reaches its maximum only after roughly 10 seconds.

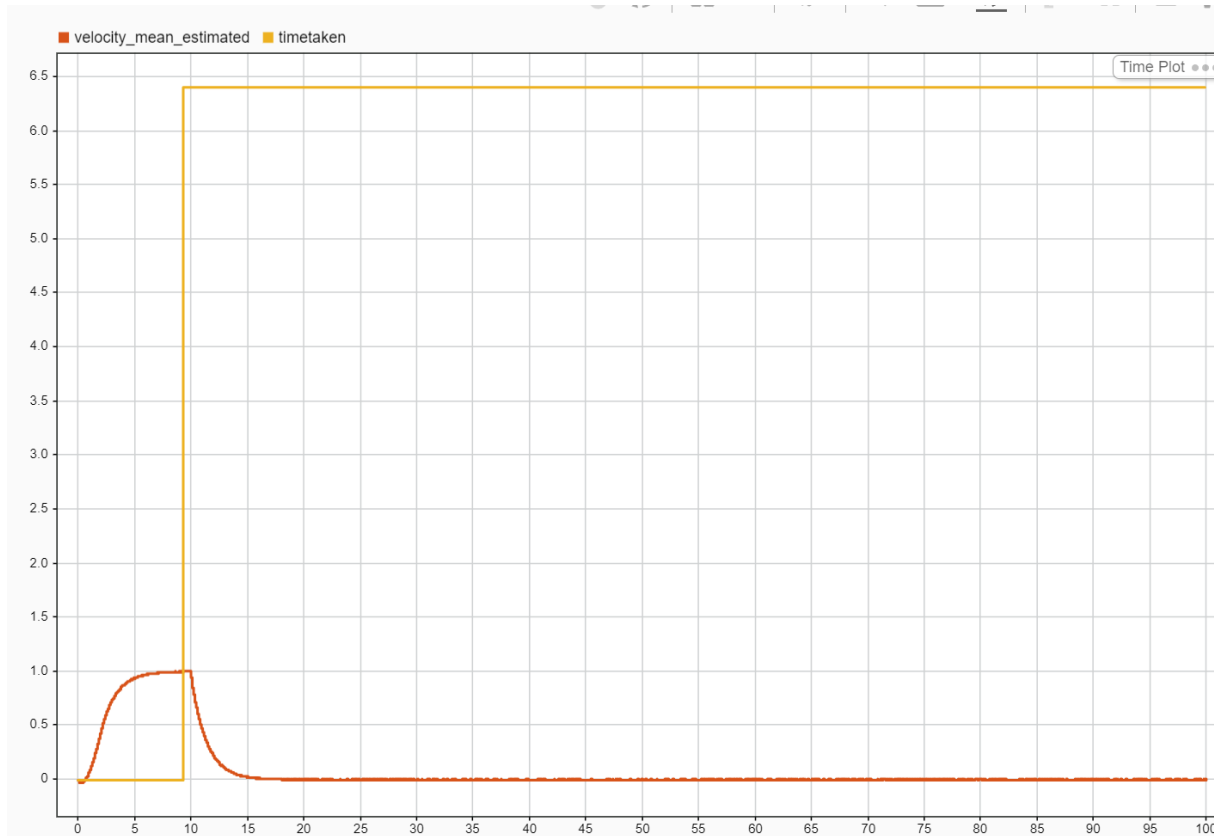


Figure 8-2 Mean estimated velocity and corresponding time marker

The plot highlights the measured traversal time. According to the simulation, the robot completes the required distance in approximately 6.4 seconds. The long acceleration phase dominates the total time, making this configuration unsuitable for competitive performance

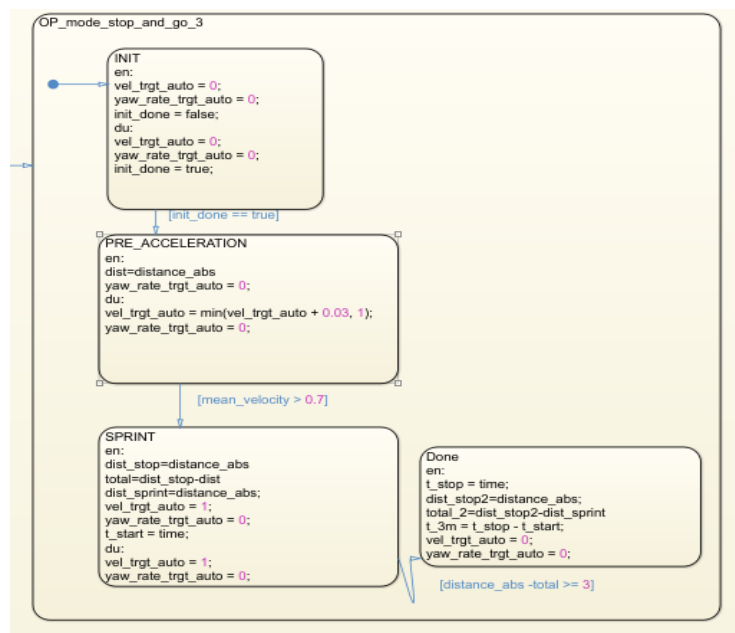


Figure 8-3 Stateflow-based phase logic for velocity ramp-up

The State flow chart defines how the robot moves through the sprint task the system does not accelerate immediately. Instead, it enters a pre-acceleration phase where the velocity target is increased gradually. Once the estimated mean velocity crosses a defined threshold, the logic switches decisively into sprint mode. The run ends not by time, but by distance

When implemented on the real-time hardware, the controller exhibits behaviour comparable to the virtual model. The robot successfully completes the three-meter task, but the motion remains conservative. The measured traversal time is approximately 7 seconds, which corresponds to low average velocity of 0.45 m/s. Although stability is maintained throughout the run, this performance does not align with the objective of the task.

Table 1:-Feedback table for velocity controllor

Control structure	Final Feedback gains	Average velocity
Velocity Control	[-3 20 1.5]	0.45m/s

8.3Position–Velocity State-Feedback Controller

The improved controller combines position-based and velocity-based control objectives to achieve stable, balanced motion across both simulation and real-time hardware. Instead of relying solely on a velocity target, the controller adapts the desired velocity based on remaining distance, allowing aggressive motion when far from the target and smoother behaviour as the robot approaches the goal.

A supervisory State flow layer schedules motion through pre-acceleration, sprint, and braking phases, while a distance-error–based velocity reference prevents excessive acceleration near the endpoint. This coupling between position error and velocity command reduces oscillations, improves robustness, and maintains balance even under aggressive operating conditions.

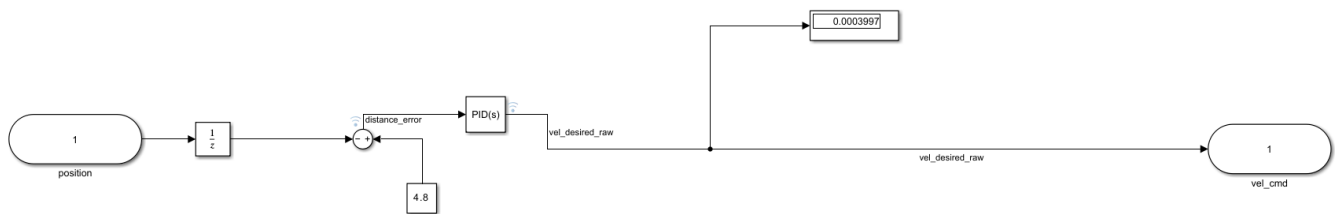


Figure 8-4 Velocity reference generation from distance error

This block generates the velocity command based on position error rather than a fixed speed reference. The current position is compared against the target distance, and the resulting distance error is processed through a PID controller to produce a desired velocity signal. By shaping the velocity as a function of position error, the controller avoids abrupt acceleration and eliminates the need for hard stopping logic. This approach results in smoother motion, improved stability, and better convergence at the target distance, particularly in real-time hardware execution.

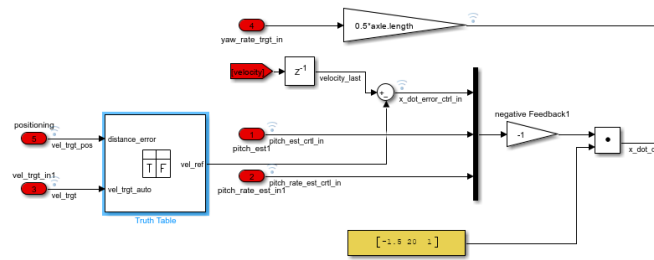


Figure 8-5 Truth-table-based velocity selection

A truth table switches between position-derived and predefined velocity targets based on distance error, ensuring appropriate speed commands during different phases of motion. This structure enables adaptive behaviour without destabilizing the core balancing controller.

Condition Table				
	DESCRIPTION	CONDITION	D1	D2
1	Example condition 1	distance_error <= 0.5	T	F
ACTIONS: SPECIFY A ROW FROM THE ACTION TABLE			1	2

Action Table		
	DESCRIPTION	ACTION
1	Example action 1 called from D1 & D2 column in condition table	vel_ref = distance_error ;
2	Example action 2 called from D3 column in condition table	vel_ref = vel_tgt_auto;

Figure 8-6 Truth table conditions

The truth table selects the velocity reference based on the current distance error. When the robot is close to the target, the reference velocity is derived directly from the distance error, enforcing smooth deceleration and precise positioning. Otherwise, a predefined velocity target is used to prioritize forward motion.

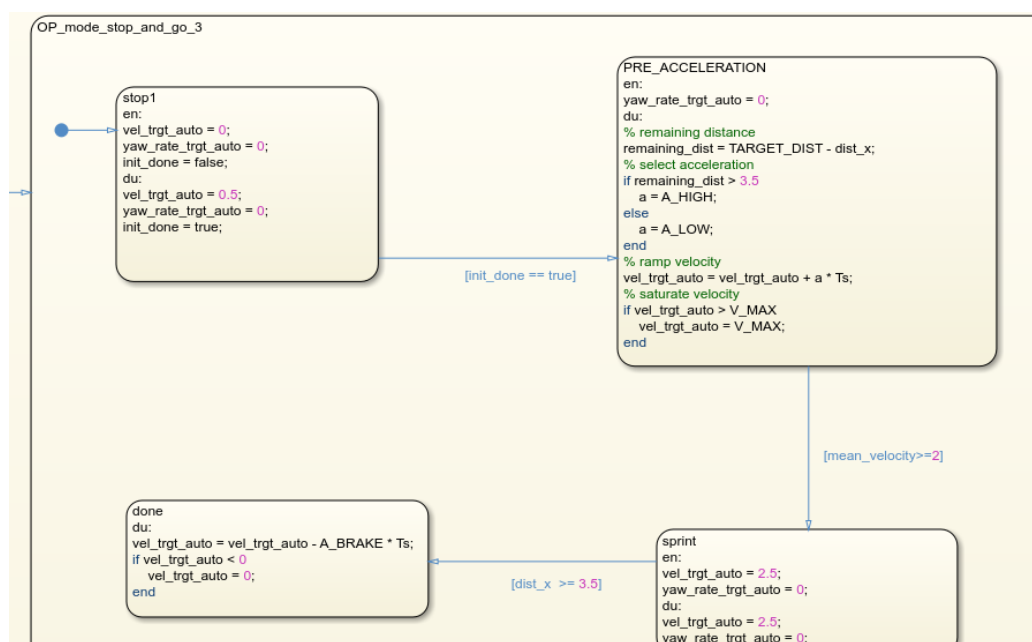


Figure 8-7 Phase-scheduled State flow logic

This State flow chart controls the robot’s motion using distinct phases. The velocity target is increased gradually, held constant during sprinting, and reduced smoothly once the target distance is reached to ensure stable stopping.

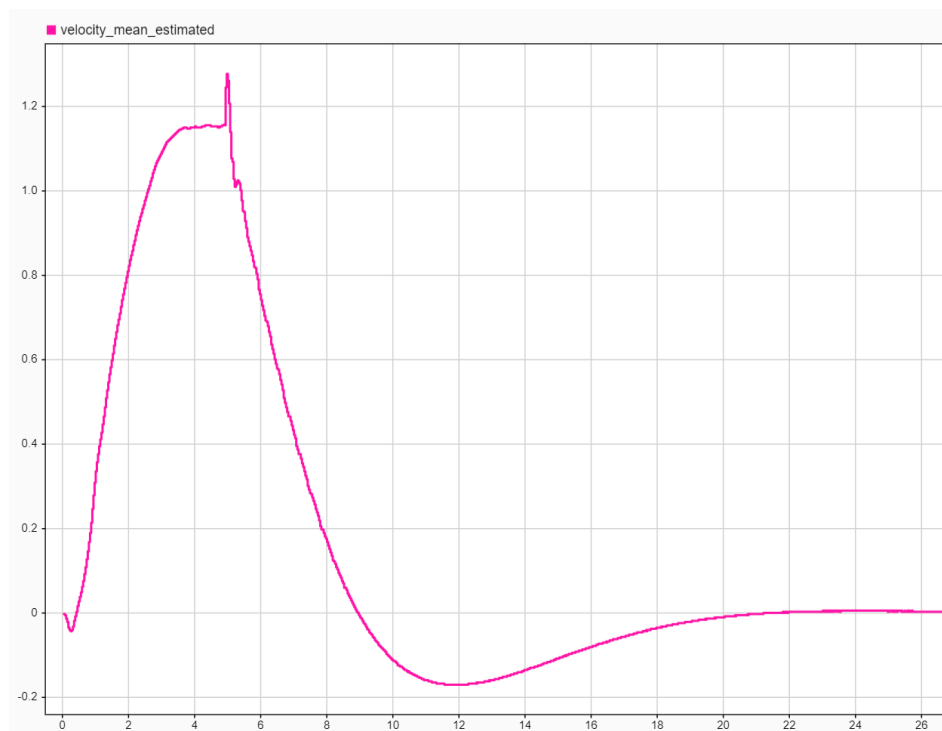


Figure 8-8 Mean forward velocity response using the position–velocity controller.

The robot accelerates to a peak velocity, followed by deceleration as the target distance is reached. Minor overshoot is observed, but the system remains stable and settles cleanly to zero velocity.

Table 2:-summary of the position–velocity state-feedback controller across multiple trials

Sr No.	controller	Gains	Trail number	Challenge time with pre acceleration Of 0.5m (sec)	Challenge time without pre acceleration (sec)	Average velocity (m/s)
1	Position–Velocity State-Feedback Controller	[-1.5 20 1]	Trail-1	2.58	2.8	1.07
2			Trail-2	2.56	2.72	1.10
3			Trail-3	2.59	2.75	1.09

Table 3:-Summary of all the controllers for challenge 1

Sr No.	Controller	Challenge time (sec)
1	State-Feedback Velocity Controller	6.5
3	Position–Velocity State-Feedback Controller	2.75

Virtual simulation : - [virtual simulation final](#)

Real time implementation : - [real time implementation of 3m sprint](#)

8.2. Challenge 2 – Driving fast with turns

In this task, the robot is pushed to move fast over a distance of 3 meters while also making two controlled turns along the way. The same core controller used for straight-line driving is kept, but it is extended to handle rotational commands so the robot can change direction without losing balance. To make this work at high speed, the forward velocity, yaw-rate targets, and control gains are carefully adjusted so the robot can turn sharply while still staying stable. The performance is judged by how accurately the robot completes the turns around the 1-meter and 2-meter marks, and by how smooth and stable its overall motion remains during the run.

8.2.1 Stateflow logic for distance- and yaw-based motion control

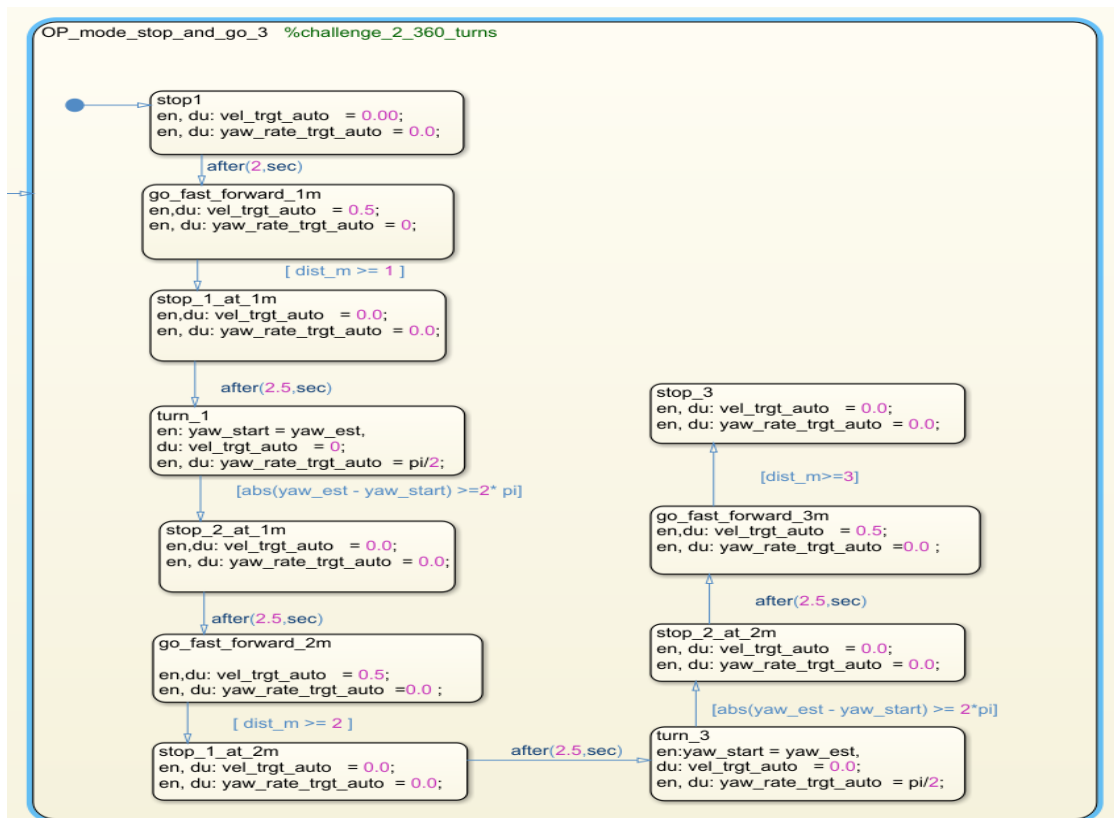


Figure 8-9 Stateflow sequence for fast forward motion with two

Stateflow chart controls how the robot moves through the task. It first accelerates forward, then stops at 1 m to perform a full 360° turn, moves forward again to 2 m, performs another 360° turn, and finally drives to 3 m before stopping. At each step, the forward speed and yaw rate are switched on or off to make sure the robot moves, turns, and settles in a controlled and stable way

```

% complementary filter here combines LP filtered pitch estimation
% based on accelerometer data and integrated HP filtered pitch estimation
% based on gyro data
%
%-----
persistent tau C pitch_from_acc_LP_filtered pitch_from_gyro_HP_filtered yaw

if isempty(tau)
    tau = 1./ (2*pi*fc);
    C = tau/(tau+dt);
    pitch_from_acc_LP_filtered = 0.0;
    pitch_from_gyro_HP_filtered = 0.0;
    yaw = 0.0;
end

% ----- PITCH ESTIMATION -----
pitch_from_acc = atan2(acc_forward_meas, acc_vertical_meas);

pit_rate_est = pitch_rate_meas * pi/18000; % [rad/s]

pitch_from_acc_LP_filtered = C*pitch_from_acc_LP_filtered ...
    +(1-C)*pitch_from_acc;

pitch_from_gyro_HP_filtered = C*(pitch_from_gyro_HP_filtered ...
    + pit_rate_est*dt);

pit_est = pitch_from_acc_LP_filtered + pitch_from_gyro_HP_filtered; % [rad]

% ----- YAW ESTIMATION -----
yaw_rate = gyro_z_meas * pi/18000; % [rad/s]
yaw = yaw + yaw_rate * dt; % integrate
yaw_est = yaw; % [rad]

end

```

Figure 8-10 Complementary filter for pitch and yaw estimation

This code combines accelerometer and gyroscope data to get a clean and stable estimate of the robot's pitch and yaw.

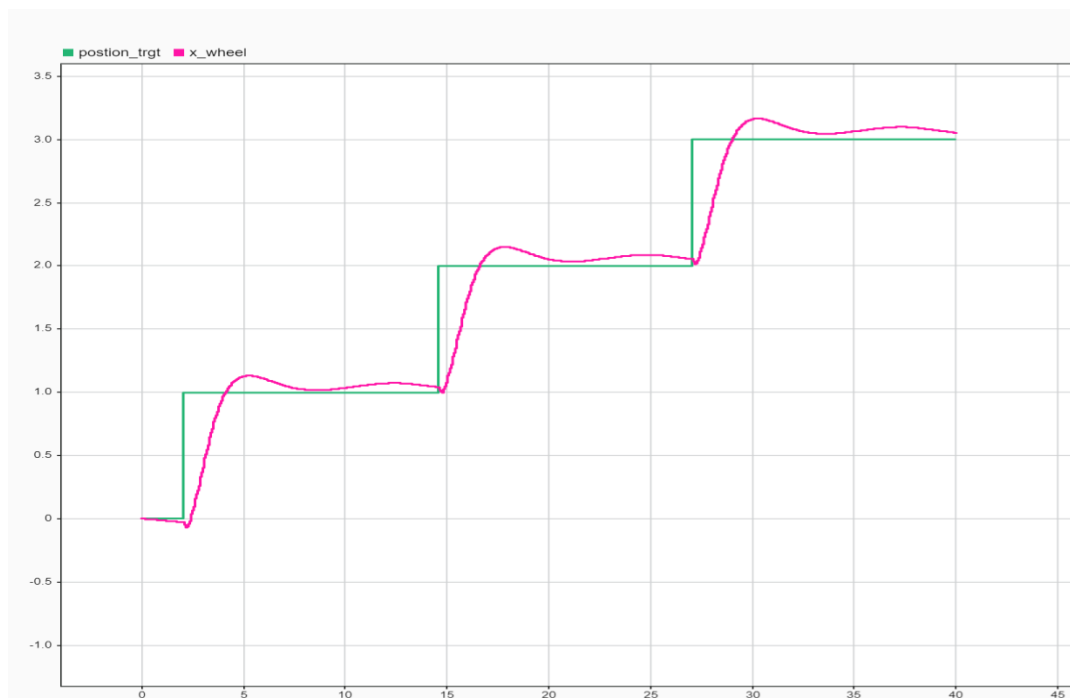


Figure 8-11 Response of the robot at different position targets

This figure shows how the robot reacts when the desired position is changed in steps. As the target jumps from one level to the next, the robot moves quickly toward it, briefly overshoots, and then settles close to the commanded position.

8.2.2 PID-based control using conditional switching

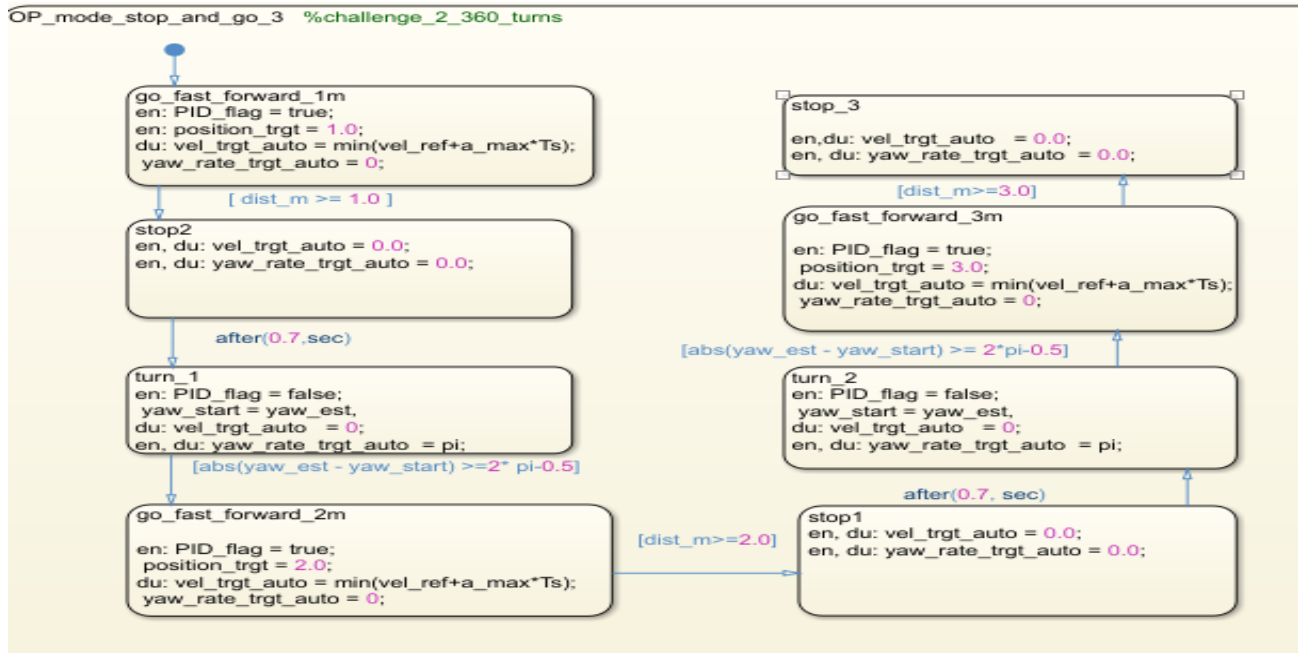


Figure 8-12 Stateflow logic for fast forward motion with two controlled 360° turns

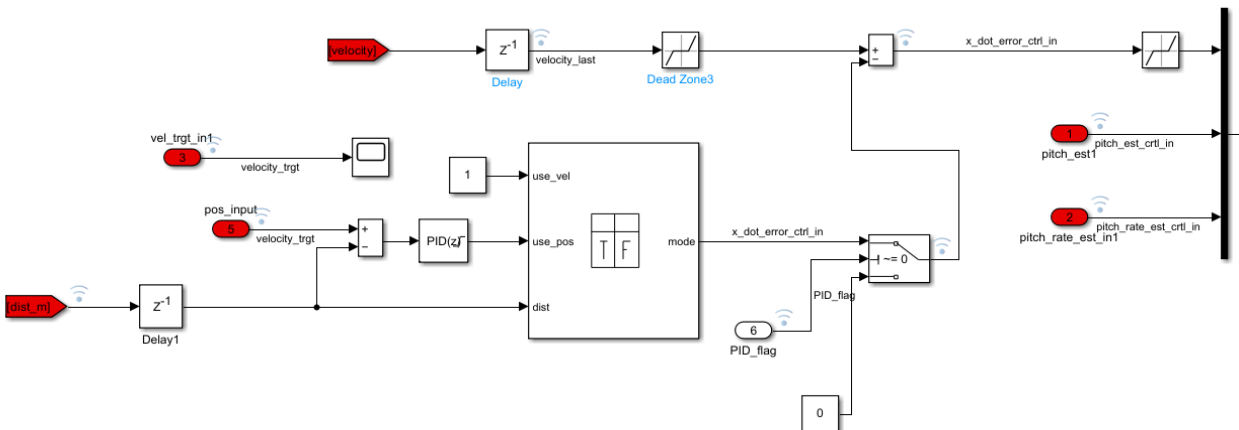


Figure 8-13 PID control that switches between driving and holding position

A position-based PID controller was added using the distance signal $dist_m$ to drive the robot toward each target and hold it there. After testing several gain settings, $K_p = 1.8$, $K_i = 0.08$, and $K_d = 0.05$ gave the best balance between fast movement and stable settling. A Stateflow switching logic was then used to smoothly switch the robot from moving to holding position once the target was reached, preventing drift and overshoot

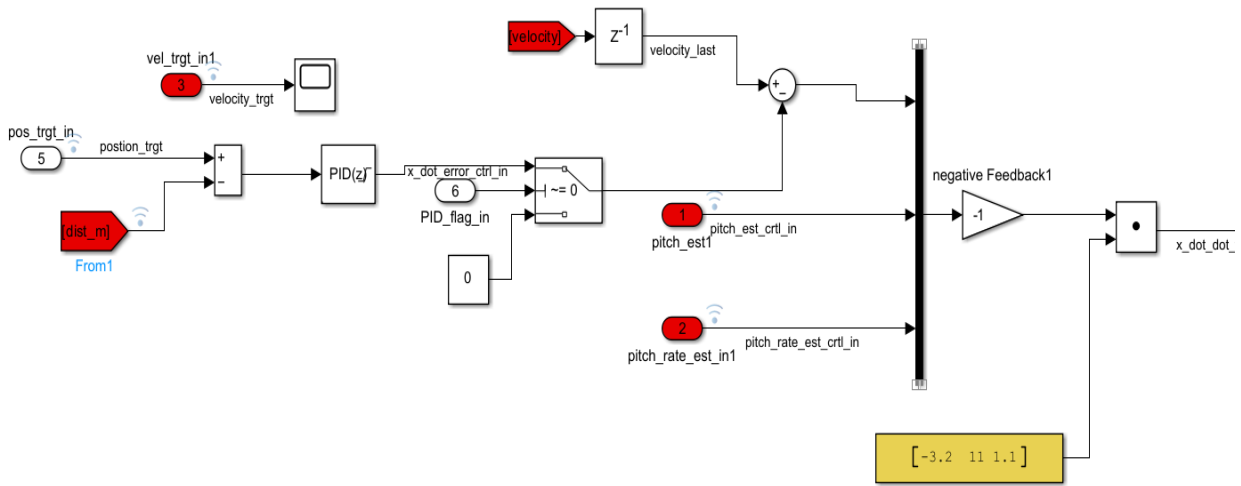


Figure 8-14 Final position PD controller with state-based switching

PD controller acts on the position error to generate a velocity correction, which is combined with the balance controller. The proportional gain ($K_p = 3.2$) provides the main restoring force toward the target, while the derivative gain ($K_d = 1.1$) helps damp motion and reduce overshoot. A logic-based switch (PID_flag) decides when the position controller should be active and when it should be ignored, so the robot can move freely when far from the target and smoothly hold its position once it arrives.

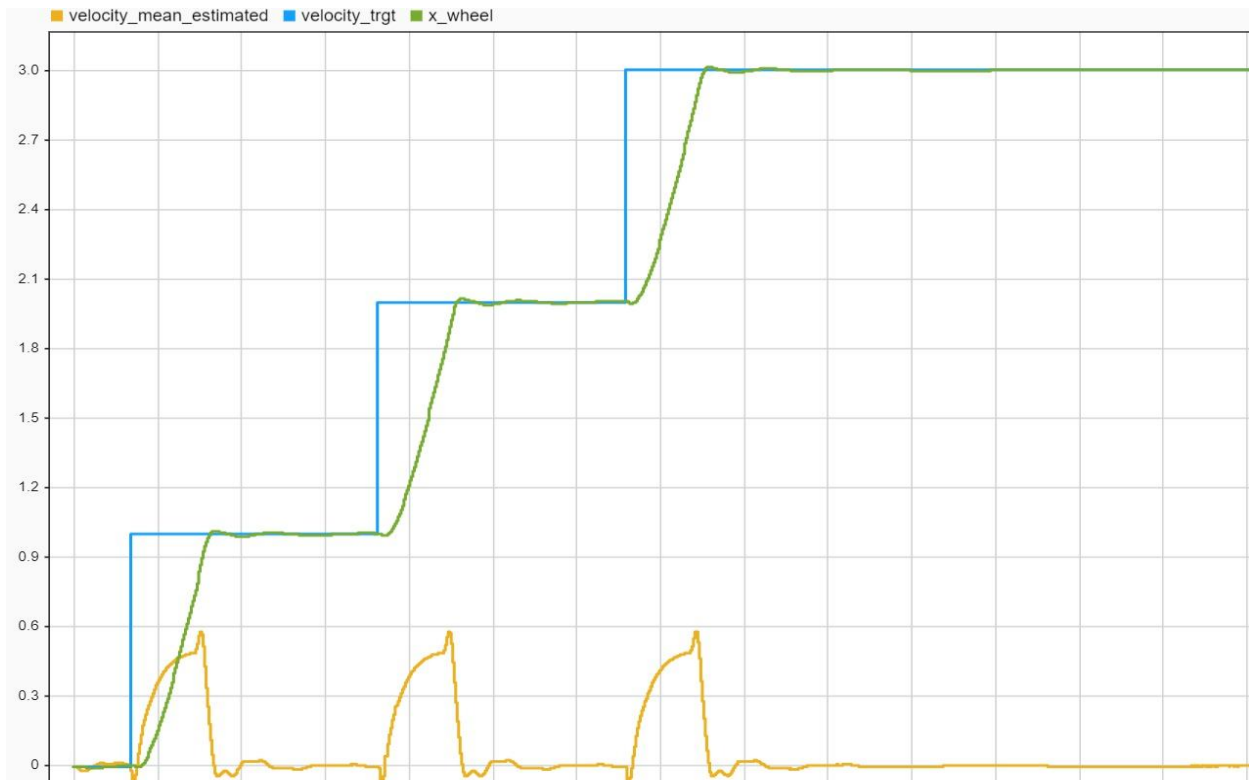


Figure 8-15 Robot response to step-wise position and velocity commands

Table 4:- Position deviation comparison for different control strategies

Sr no	Control Strategy	Deviation at 1 meter	Deviation at 2 meters	Deviation at 3 meters	Angular Deviations 1 meter	Angular Deviations 1 meter
1	Stateflow logic for distance- and yaw-based motion control	25cm	30cm	35cm	6	13
2	PID Controller	5cm	7cm	10cm	3	8
3	PD controller	1cm	2cm	2cm	≈ 0	≈ 0

Realtime implemetation: - [realtime implementation for Driving fast with turns](#)

8.3. Challenge 3 – Additional weight compensation

The aim of this task is to make sure the robot holds its ground even when extra weight is suddenly placed on it. As soon as a load is added, the robot instinctively rolls toward it in order to keep its balance, and that movement shifts it away from its original position. To counter this behaviour, the controller is built around a zero-position reference, so once the robot regains stability it is actively driven back to where it started. How well this works is judged by three things: how fast the robot settles back, how far it overshoots the target, and how much it dips below it before stabilizing again

8.3.1. Steady-state error control

To solve this task, three different control approaches were tested, each one improving on what the previous one could not handle. The first step was to deal with the steady-state drift that appears when extra weight is added to the robot. Because the added mass creates a constant torque, the robot keeps leaning and slowly rolling away even though it looks balanced. By adding an integral term to the pitch control loop, this constant bias could be cancelled, allowing the robot to stand upright without drifting. While this fixed the long-term offset, it did not guarantee that the robot would return cleanly to its original position after being disturbed.

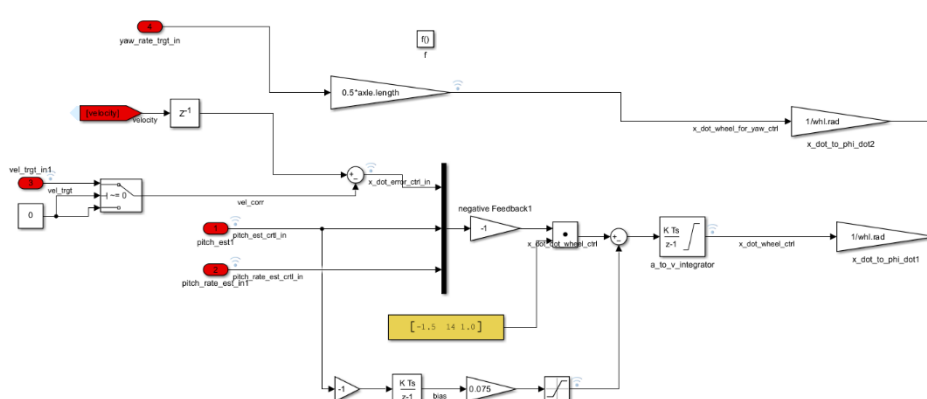


Figure 8-16 Design of the steady-state error compensation controller

8.3.2. PID controller implementation

To improve position recovery, a position PID controller was added so the robot could actively drive itself back to zero after a load was applied. This allowed the robot to recover its position even under a heavy load of 650 g, which was built up using everyday objects like coins and keys. However, this came with a clear drawback. When the weight was added, the robot overshot its starting point by about 12–15 cm in both directions and began to oscillate strongly. Instead of settling quickly, it took roughly 23 seconds to return to its original position, making the response slow and inefficient. While the PID controller proved that the robot could handle large disturbances, the large overshoot and long settling time showed that this approach alone was not sufficient for a fast and well-controlled recovery.

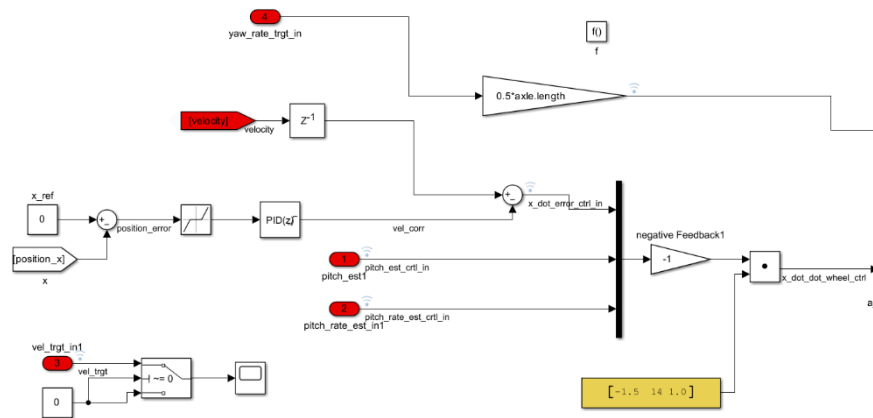


Figure 8-17 Position error PID control architecture

8.3.3. Cascaded PID control

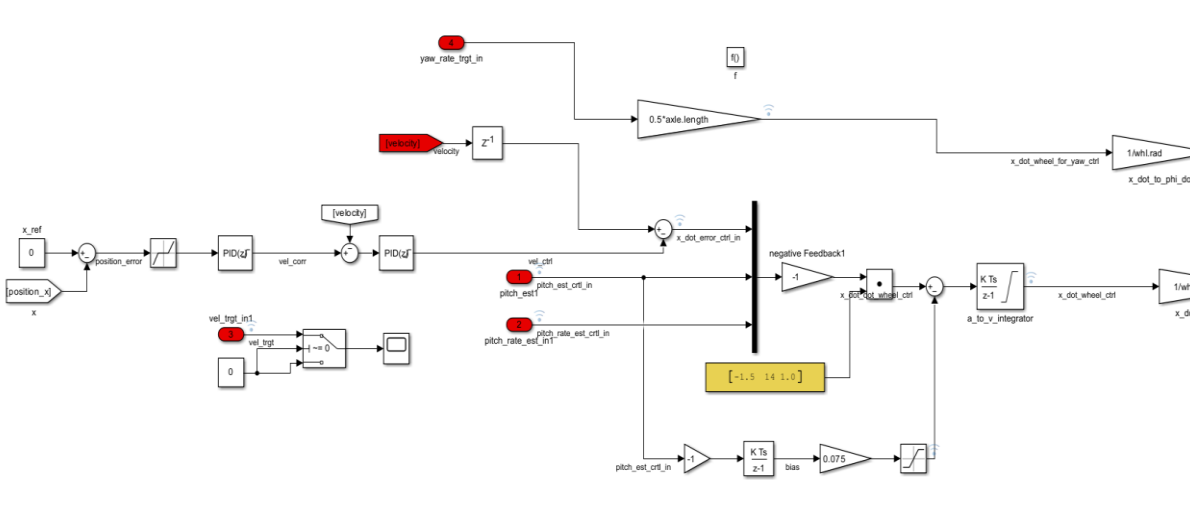


Figure 8-18 Cascaded PID structure for position and velocity regulation

To improve this behaviour, a cascaded PID structure was finally introduced. In this setup, one controller handled position while another handled velocity, allowing slow position corrections and fast motor dynamics to be controlled separately. This made the response much smoother and more stable, with less

oscillation and a faster, cleaner return to the original position. In practice, this approach gave the best balance between stability, accuracy, and robustness when the robot was loaded with additional weight.

The physical robot performed very well with the cascaded PID controller. It settled quickly, showed almost no oscillations, and reached zero steady-state error while staying stable over time. After the first weight was added, the robot returned to its original position (0) in 5 seconds. With the second weight group, it took 7.3 seconds, and with the final weight group it recovered in 5.2 seconds.

When the first weight was attached, the robot initially shifted forward by about +32 cm, but it quickly corrected itself. For all later weights, the position stayed within a ± 10 cm tolerance while balancing. The controller also proved to be highly robust: across repeated trials, the robot behaved the same way each time, showing stable and repeatable performance.

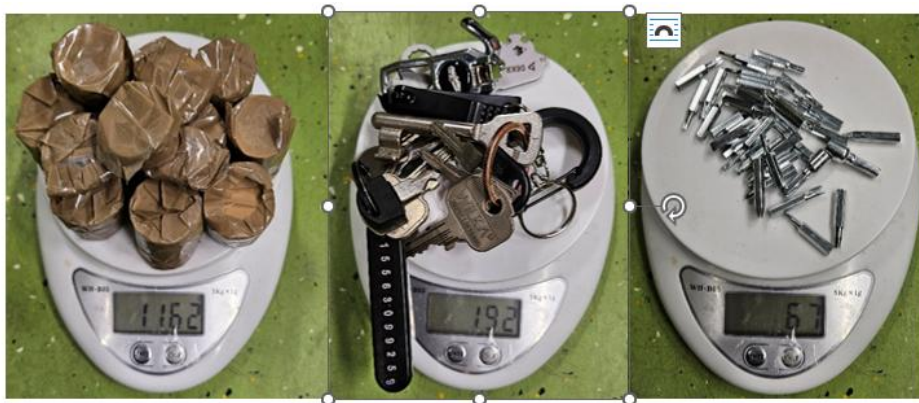


Figure 8-19 Test loads used for balance experiments

Table 5:- Cascaded PID gains and the corresponding balanced weights on both sides

	Kp	Ki	Kd	Gains	Weight total (g)
Position PID	1.5	0.9	0.2	[-1.5 14 1]	1421
Velocity PID	2.5	0	0.17		

Real time implementation:- [realtime implementing adding weights on two sides](#)

When extra weight was added to only one side of the robot, it stayed balanced up to 919 g before it finally failed. As seen in the video, the robot began to vibrate and shake because of pitch control errors. These small oscillations kept growing, which reduced stability and eventually caused the robot to fall. With better tuning of the pitch controller, the robot should be able to handle even more uneven weight without losing balance

Table 6:-Cascaded PID gains and balanced weight with load applied on one side

	Kp	Ki	Kd	Gains	Weight total (g)
Position PID	0.6	5	0	[-3 10 1]	919
Velocity PID	1.1	0	0.2		

Real time implementation: - [realtime implementation of weight balance on oneside](#)

8.4 Driving a figure 8

The goal of this challenge was to make the robot drive a figure-8 path, starting with a left turn, then a full right circle, and finishing with another left turn, all using the same turning radius. Unlike straight driving, this motion keeps changing direction and curvature, which made the robot much harder to control. While trying to follow the path, the robot had to stay balanced, keep a steady forward speed, and rotate at the right rate at the same time. Several control setups were tested and adjusted until the robot could complete the full figure-8 without losing stability or drifting off the path.

8.4.1 Initial approach: time-based figure-8 motion

The first attempt at the figure-8 task used a simple time-based Stateflow sequence. The idea was not to be accurate yet, but to check whether the robot could even trace a figure-8 shape using basic motion commands. At this stage, no feedback from position or yaw was used the robot was told how fast to move and how long to turn, and it simply followed those commands. The figure-8 was created by chaining together circular motions, all starting from the same reference point. Each part of the motion ran for a fixed amount of time, and the yaw rate was calculated from the chosen forward speed and turning radius.

Table 7:-Parameters and step-by-step calculation for circular motion

Sr no	Quantity	Symbol	Formula	Value	Unit
1	Forward velocity	v	-	0.25	m/s
2	Turning radius	r	-	0.50	m
3	Yaw rate	ω	$\omega = v / R$	0.50	Rad/sec
4	Angular travel (full circle)	θ	$\theta = 2\pi$	6.283	rad
5	Time for one full circle	t	$T = \theta / \omega$	12.56	s

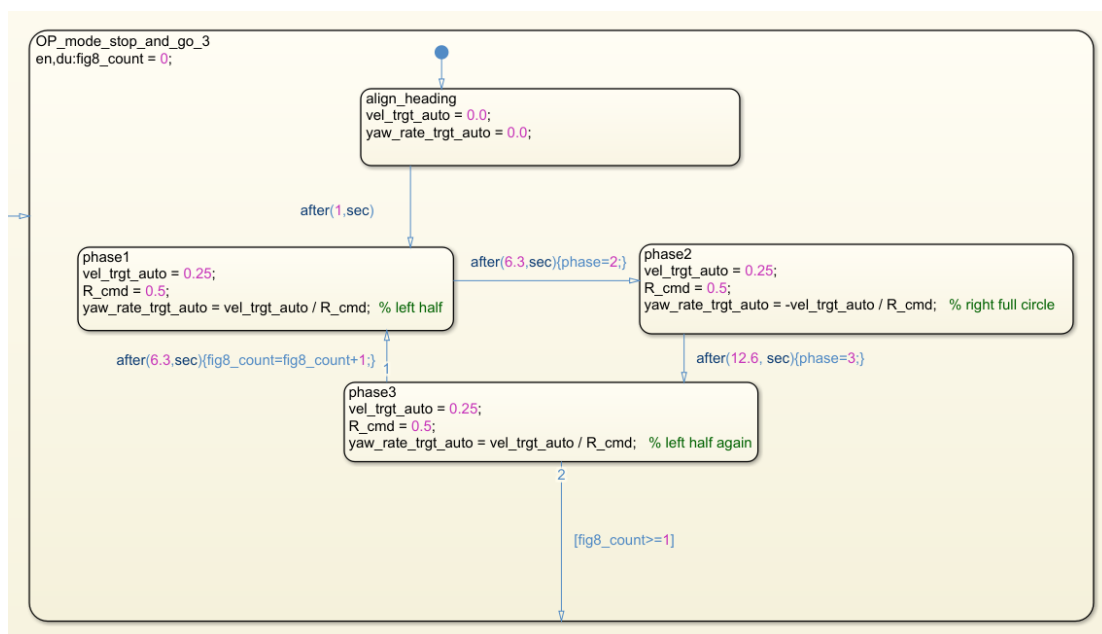


Figure 8-20 Stateflow-based time-driven logic for the figure-8 path

In simulation, this approach worked well. Using the Simscape Mechanics Explorer, the robot appeared to follow a clean and smooth figure-8 path. However, when the same logic was run on the real robot, the motion no longer matched the simulation. Friction, motor limits, sensor noise, and small timing errors all began to accumulate, causing the turns to stretch or shrink and the trajectory to slowly drift out of shape.

Table 8:-Impact of forward velocity and timing on the figure-8 path

Iteration	Forward Velocity (m/s)	Radius (m)	Yaw Rate Definition	Total Figure 8 Time (s)
1	0.20	0.5	$\omega=v/R$	~31.4
2	0.25	0.5	$\omega=v/R$	~26.2
3	0.30	0.5	$\omega=v/R$	~20.9

8.4.2 Second iteration: yaw-driven figure-8 control

In the second version of the figure-8 controller, the focus was shifted from time-based motion to direct yaw control. The earlier approach worked in simulation but failed on the real robot because small changes in speed and timing quickly distorted the path. To fix this, the yaw motion was explicitly regulated so the robot would turn with a consistent curvature even if its forward speed changed slightly.

```

persistent tau C pitch_from_acc_LP_filtered pitch_from_gyro_HP_filtered yaw

if isempty(tau)
    tau = 1./ (2*pi*fc);
    C = tau/(tau+dt);
    pitch_from_acc_LP_filtered = 0.0;
    pitch_from_gyro_HP_filtered = 0.0;
    yaw = 0.0;
end

% ----- PITCH ESTIMATION -----
pitch_from_acc = atan2(acc_forward_meas, acc_vertical_meas);

pit_rate_est = pitch_rate_meas * pi/18000; % [rad/s]

pitch_from_acc_LP_filtered = C*pitch_from_acc_LP_filtered ...
    +(1-C)*pitch_from_acc;

pitch_from_gyro_HP_filtered = C*(pitch_from_gyro_HP_filtered ...
    + pit_rate_est*dt);

pit_est = pitch_from_acc_LP_filtered + pitch_from_gyro_HP_filtered; % [rad]

% ----- YAW ESTIMATION -----
yaw_rate = gyro_z_meas * pi/18000; % [rad/s]
yaw = yaw + yaw_rate * dt; % integrate
yaw_est = yaw; % [rad]

end

```

Figure 8-21 Complementary-filter-based pitch and yaw estimation code

During the figure-8 motion, turning was controlled by switching the sign of the yaw rate in the Stateflow logic rather than relying on position tracking. This made the transitions between left and right turns smoother and reduced sideways drift compared to the time-based method. Overall, this yaw-based approach produced more repeatable trajectories and was less sensitive to timing errors.

8.4.3 Final iteration: figure-8 control using position feedback

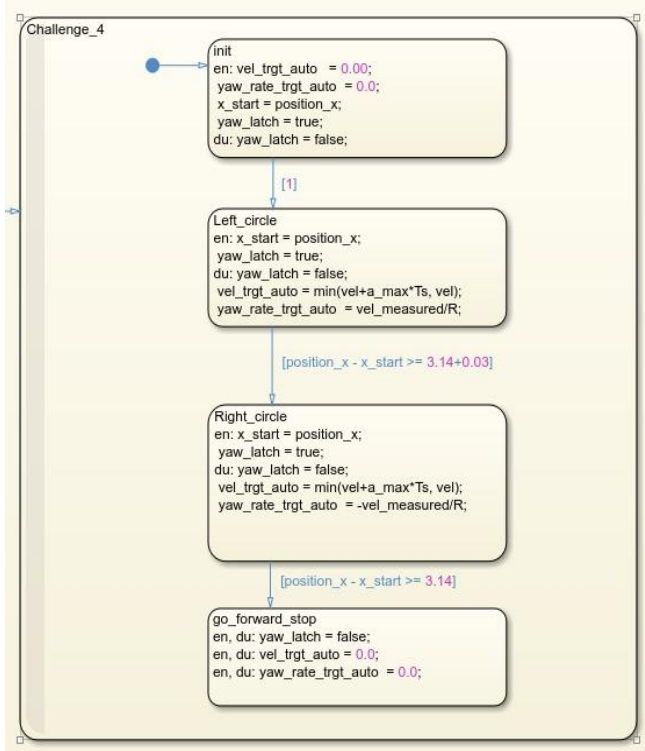


Figure 8-23 Position-referenced Stateflow logic for figure-8 trajectory

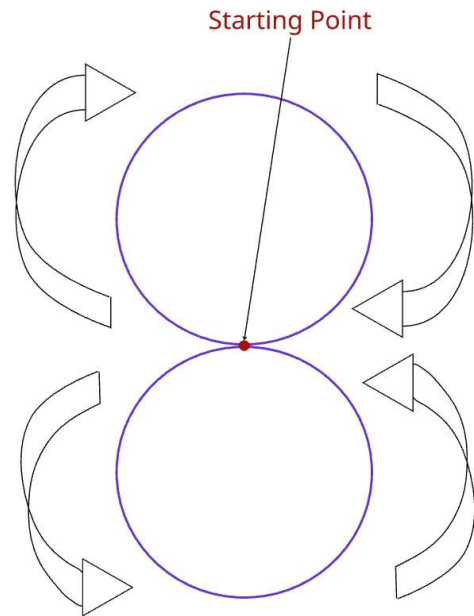


Figure 8-22:-configuration for figure 8

This Stateflow chart controls the figure-8 motion using position feedback instead of time. The robot starts by recording its current position, then drives a left circle until it reaches a defined distance, switches to a

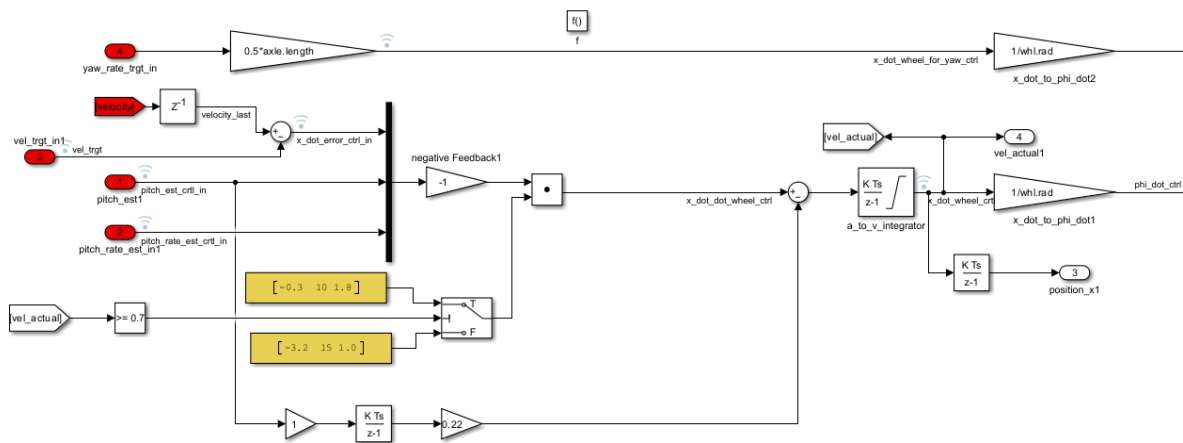


Figure 8-24 final controller for driving figure 8

right circle by reversing the yaw rate, and finally moves forward to complete the pattern. By basing the transitions on measured position rather than elapsed time, the robot can follow a much more consistent and repeatable figure-8 path.

To keep the robot stable across a wide range of speeds, a velocity-dependent gain switching scheme was introduced. The robot's forward speed is continuously monitored, and the controller switches between two different gain sets based on this value. At low speeds, a more responsive gain set is used to keep the robot tightly balanced. Once the speed exceeds 0.7 m/s, the controller switches to a more heavily damped setting, which reduces pitch oscillations and keeps the robot stable during fast motion. This allows the robot to move smoothly at low speeds while still remaining well controlled when driving faster through the figure-8.

The forward velocity target is not applied as a sudden step. Instead, it is ramped up using a limited acceleration,

$$V_{\text{trgt}} = \min (V_{\text{max}} + a_{\text{max}} \times T_{\text{sample}}, V_{\text{max}})$$

where a_{max} defines how quickly the robot is allowed to speed up and T_{sample} is the controller update time. This prevents sharp velocity jumps and keeps the motion smooth during both straight and curved segments.

The yaw-rate target is then calculated from the *measured* forward velocity using

$$\omega_{\text{trgt}} = \frac{V_{\text{measured}}}{R}$$

rather than from the commanded speed. This way, if friction, load changes, or motor limits cause the robot to move slower or faster than expected, the yaw rate adjusts automatically. As a result, the robot keeps a nearly constant turning radius instead of drifting or tightening its turns.

By basing the turns on measured motion instead of fixed timing, this yaw-driven approach removed one of the biggest sources of error in the earlier designs and produced much more consistent and repeatable figure-8 trajectories on the real robot.

Table 9:- Evaluation of motion control approaches for the figure-8 trajectory

Controller Type	Type of Motion Executed	Percentage of Task Completed	Time (s)
Position Controlled	Left and Right Full Circles	100	8.6
Position Controlled	Combination of semicircle followed by circle	100	11

Real time simulation: - [simulation_for_figure_8](#)

8.5. DIP switch control

To make it easier to change operating modes without constantly rebuilding and flashing the model, DIP switches were added to the system. A Simulink function called `Dip_Switch` reads the switch states and selects the active mode. The four DIP switches are connected to the Arduino's A1–A4 analog input pins and are read through the MecRoka AnalogRead block, allowing the robot's behaviour to be changed instantly by flipping the switches. The raw analog readings are scaled using a gain of 5/1023, so an active

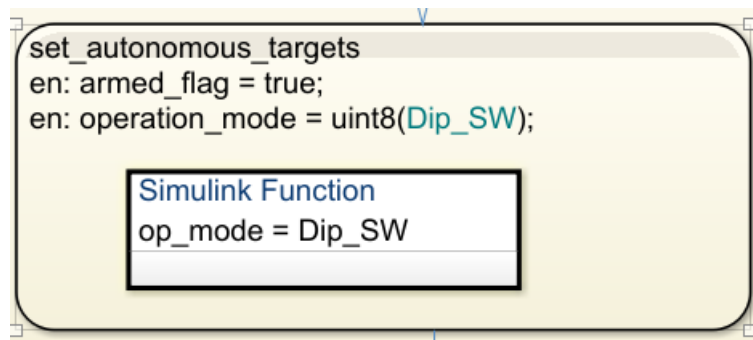


Figure8-25. Simulink-based DIP switch interface for mode control

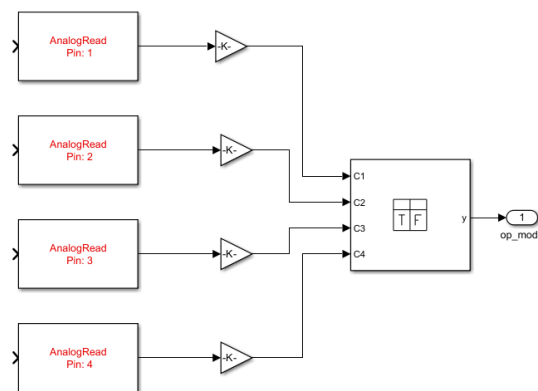


Figure 8-26. Truth-table based mode selection in Simulink

DIP switch produces a value close to 5 and an inactive one gives 0. This makes it easy to detect the selected mode while allowing the model to keep running without needing constant firmware reuploads. The scaled outputs from each gain block are fed into a truth table, which determines the active operating mode based on the DIP switch settings. A fail-safe is included so that if an invalid switch combination is detected, the system automatically falls back to the default balancing mode (Mode 1) to keep the robot stable.

Condition Table

	DESCRIPTION	CONDITION	D1	D2	D3	D4	D5
1	Switch_1	$C1 \geq 4.9$	T	F	F	F	-
2	Switch_2	$C2 \geq 4.9$	F	T	F	F	-
3	Switch_3	$C3 \geq 4.9$	F	F	T	F	-
4	Switch_4	$C4 \geq 4.9$	F	F	F	T	-
		ACTIONS: SPECIFY A ROW FROM THE ACTION TABLE	1	2	3	4	5

Action Table

	DESCRIPTION	ACTION
1	Challenge 1	$y = 2;$
2	Challenge 2	$y = 3;$
3	Challenge 3	$y = 4;$
4	Challenge 4	$y = 5;$
5	default	$y = 1;$

Figure 8- 27 Logic table for selecting the operating mode

The scaled outputs from the gain blocks are fed into a truth table, which selects the active operating mode based on the DIP switch settings. A fail-safe is included so that if an invalid switch combination is detected, the system automatically falls back to the default balancing mode (Mode 1) to keep the robot stable.

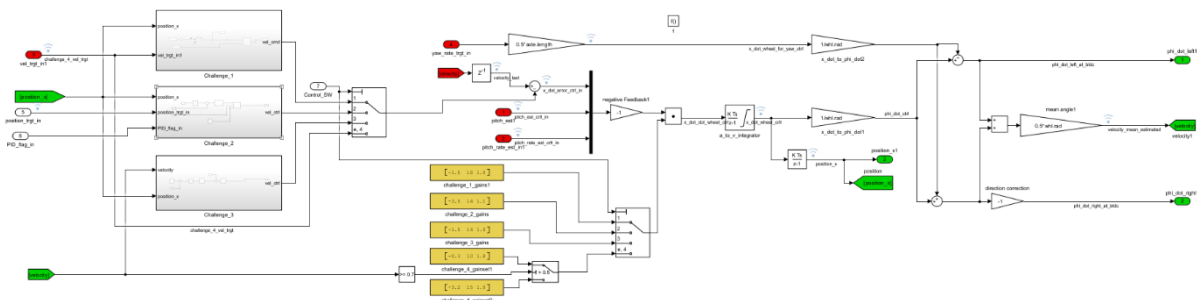


Figure 8-28 Combined controller design for all 4 challenges with control switch for logic selection

Table 10:-Use of AI tools in this project

Area	How AI was used	What AI provided (ChatGPT)	HOW I IMPLEMENTED
Controller design	Discussed different control architectures (time-based, yaw-based, position-based, cascaded PID, PD with switching	Comparison of different design options	Implemented and tuned the controllers in Simulink
Gain tuning strategy	Helped reason about stability, overshoot, and oscillation	Guidelines on how gains affect behavior	Performed tuning through simulation and hardware testing
Stateflow logic	Explained switching logic and mode transitions	Logic ideas and structure	Built the Stateflow charts and verified them on the robot
Report writing	Rephrased, clarified, and improved text and Helped write captions and explanations	Language polishing and structure(used ChatGPT and quill bot)	Provided raw content, results, and technical meaning